

Лабораторная работа №5 – «Компилятор GCC. Оптимизация»

1. Введение

Актуальность: Оптимизация кода критически важна для производительности приложений. Компилятор GCC предоставляет различные уровни оптимизации, влияющие на скорость выполнения и размер программы.

Цель: Исследование влияния различных опций оптимизации компилятора GCC на производительность и размер исполняемого файла.

Задачи:

- 1) Разработать тестовую программу
- 2) Создать сценарий для автоматического тестирования оптимизаций
- 3) Проанализировать результаты различных уровней оптимизации

2. Методика исследования

Инструментальная платформа:

ОС: Ubuntu Linux 24.04.3

Компилятор: GCC 13.3.0

Процессор: AMD Ryzen 5 3500U with Radeon Vega Mobile Gfx

Тестовая программа: test.cpp (вычисление суммы степеней чисел с плавающей точкой)

3. Результаты (часть 1)

Тестирование оптимизаций для файла: test.cpp

=====

Текущая опция оптимизации: -O0

Время выполнения:

real 0m9,917s

user 0m9,891s

sys 0m0,027s

Размер исполняемого файла:

16440

=====

Текущая опция оптимизации: -Os

Время выполнения:

real 0m7,857s

user 0m7,831s

sys 0m0,027s

Размер исполняемого файла:

16232

=====

Текущая опция оптимизации: -O1

Время выполнения:

real 0m3,333s

user 0m3,331s

sys 0m0,002s

Размер исполняемого файла:

16232

=====

Текущая опция оптимизации: -O2

Время выполнения:

real 0m2,200s

user 0m2,186s

sys 0m0,015s

Размер исполняемого файла:

16232

=====

Текущая опция оптимизации: -O3

Время выполнения:

real 0m1,793s

user 0m1,777s

sys 0m0,016s

Размер исполняемого файла:

16256

=====

Текущая опция оптимизации: -O2 -march=native

Время выполнения:

real 0m1,929s

user 0m1,918s

sys 0m0,012s

Размер исполняемого файла:

16232

=====

Текущая опция оптимизации: -O3 -march=native

Время выполнения:

real 0m1,839s

user 0m1,834s

sys 0m0,004s

Размер исполняемого файла:

16256

=====

Текущая опция оптимизации: -O2 -march=native -funroll-loops

```

Время выполнения:
real  0m1,865s
user  0m1,850s
sys   0m0,014s
Размер исполняемого файла:
16232
=====
Текущая опция оптимизации: -O3 -march=native -funroll-loops
Время выполнения:
real  0m1,866s
user  0m1,826s
sys   0m0,038s
Размер исполняемого файла:
16256

```

4. Анализ полученных результатов (часть 1)

Опции оптимизации	Время выполнения	Размер исполняемого файла
-O0	real 0m9,917s user 0m9,891s sys 0m0,027s	16440
-Os	real 0m7,857s user 0m7,831s sys 0m0,027s	16232
-O1	real 0m3,333s user 0m3,331s sys 0m0,002s	16232
-O2	real 0m2,200s user 0m2,186s sys 0m0,015s	16232
-O3	real 0m1,793s user 0m1,777s sys 0m0,016s	16256
-O2 -march=native	real 0m1,929s user 0m1,918s sys 0m0,012s	16232
-O3 -march=native	real 0m1,839s user 0m1,834s sys 0m0,004s	16256
-O2 -march=native -funroll-loops	real 0m1,865s user 0m1,850s	16232

	sys 0m0,014s	
-O3 -march=native -funroll-loops	real 0m1,866s user 0m1,826s sys 0m0,038s	16256

Вывод: Лучшее время работы – **1,793s** (соответствует уровню оптимизации **-O3**)

5. Результаты (часть 2)

```
=====
Текущая опция оптимизации: -O3 -fipa-pta -flto
Время выполнения:
real 0m1,743s
user 0m1,730s
sys 0m0,013s
Размер исполняемого файла:
16184
=====
Текущая опция оптимизации: -O3 -fipa-bit-cp -flto
Время выполнения:
real 0m1,686s
user 0m1,632s
sys 0m0,055s
Размер исполняемого файла:
16184
=====
Текущая опция оптимизации: -O3 -fipa-vrp -flto
Время выполнения:
real 0m1,736s
user 0m1,727s
sys 0m0,010s
Размер исполняемого файла:
16184
=====
Текущая опция оптимизации: -O3 -fipa-icf -flto
Время выполнения:
real 0m1,719s
user 0m1,711s
sys 0m0,008s
Размер исполняемого файла:
16184
=====
```

Текущая опция оптимизации: -O3 -fprofile-generate
Время выполнения:
real 0m2,106s
user 0m2,069s
sys 0m0,037s
Размер исполняемого файла:
32104
=====
Текущая опция оптимизации: -O3 -fprofile-use
Время выполнения:
real 0m1,798s
user 0m1,787s
sys 0m0,011s
Размер исполняемого файла:
16288

6. Анализ полученных результатов (часть 2)

Опции оптимизации	Время выполнения	Размер исполняемого файла
-O3 -fipa-pta -flto	real 0m1,743s user 0m1,730s sys 0m0,013s	16184
-O3 -fipa-bit-cp -flto	real 0m1,686s user 0m1,632s sys 0m0,055s	16184
-O3 -fipa-vrp -flto	real 0m1,736s user 0m1,727s sys 0m0,010s	16184
-O3 -fipa-icf -flto	real 0m1,719s user 0m1,711s sys 0m0,008s	16184
-O3 -fprofile-generate	real 0m2,106s user 0m2,069s sys 0m0,037s	32104
-O3 -fprofile-use	real 0m1,798s user 0m1,787s sys 0m0,011s	16288

Вывод: Лучшее время работы – **1,686s** (соотв. опции **-O3 -fipa-bit-cp -flto**)

7. Вопросы по работе

1. Что даёт опция -march=native?

Опция `-march=native` указывает компилятору генерировать код, оптимизированный для процессора, на котором происходит компиляция. Она определяет архитектуру текущего процессора, включает доступные инструкции (SSE, AVX и др.), оптимизирует код под конкретные возможности CPU, улучшает производительность за счёт использования расширенных инструкций.

2. Почему иногда время после оптимизации становится нулевым?

Удаление «мёртвого кода» (Dead Code Elimination) (компилятор удаляет фрагменты кода, которые не влияют на результат программы: код, который не исполняется ни при каких условиях, и код, изменяющий значения переменных, которые никогда не используются).

3. Какие изменения в исполняемый код приложения вносит опция -fprofile-generate?

Инструментирование кода (добавление счётчиков выполнения), создание структур данных для сбора статистики (счётчики ветвлений, счётчики вызовов функций, время выполнения блоков кода), увеличение размера исполняемого файла, замедление выполнения из-за накладных расходов на сбор статистики.

4. Почему не имеет смысла замерять время работы приложения, собранного с опцией -fprofile-generate?

Накладные расходы (инструментирование замедляет выполнение на 20-50%), искажённые результаты (время не отражает реальную производительность), цель инструментированной версии – сбор статистики, а не измерение скорости.

5. На каком этапе порождается профиль приложения и какое расширение он имеет?

- Компиляция с `-fprofile-generate` -> создаются файлы `.gncs`
 - Запуск программы -> создаются файлы `.gcda`
 - Перекомпиляция с `-fprofile-use` -> используется собранный профиль
- `.gcno` (GCC Notes) – информация о структуре программы
- `.gcda` (GCC Data) – собранная статистика выполнения